

NEXUS

BizIQ : AI-Powered SME Decision Intelligence Platform

QUALITY ASSURANCE TESTING DOCUMENTATION

(QATD)

UMHackathon 2026

umhackathon@um.edu.my

Table of Content

Document Control	3
1. Scope & Requirements Traceability	3
1.1 In-Scope Core Features	3
1.2 Out-of-Scope	4
2. Risk Assessment & Mitigation Strategy	4
3. Test Environment & Execution Strategy	5
<i>4.1 Integration Thresholds (Merging to Main)</i>	7
5. Test Case Specifications (Drafts)	8
6. AI Output & Boundary Testing (Drafts)	11

Document Control

Field	Detail
System Under Test (SUT)	<i>Team Nexus</i>
Team Repo URL	<i>https://github.com/nloqmanhn05/BizIQ</i>
Project Board URL	<i>https://github.com/nloqmanhn05/BizIQ</i>
Live Deployment URL (optional for preliminary round)	<i>Not deployed yet (Preliminary Round)</i>

Objective:

The primary objective is to ensure that BizIQ can reliably handle business data ingestion (Excel/CSV), render accurate KPI dashboards, deliver AI-powered business insights grounded solely on uploaded data using the ilmu-glm-5.1 model, execute financial simulations, and provide smart decision recommendations all while meeting performance, security, and reliability thresholds under CI/CD checkpoints.

1. Scope & Requirements Traceability

1.1 In-Scope Core Features

- **Advanced Dashboard:** Visualize business health with KPI metrics (Revenue, Gross Margin, YoY Growth) via high-fidelity charts using uploaded data
- **AI Business Assistant:** Streaming AI chat interface (ilmu-glm-5.1) grounded on uploaded business data to provide contextual expert advice
- **Financial Simulator:** Run “What-If” financial scenarios by adjusting pricing, COGS, or marketing spend to project bottom-line impact

1.2 Out-of-Scope

1. User Authentication & Account Management
2. Mobile Responsiveness & Cross-browser Compatibility
3. User Profile Settings & Preferences

2. Risk Assessment & Mitigation Strategy

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
AI returns hallucinated business insights not grounded on uploaded data	4	5	20 (Critical)	Strict system prompt grounding: AI is constrained to only reference uploaded dataset context. RAG-style context injection before each prompt. [Context window strictly scoped to parsed Excel/CSV data only]	Feed AI a prompt about a product not in the dataset and verify the model refuses to fabricate. Log and compare AI output against known ground truth from mock dataset. [Cross-validate AI output against pre-computed expected values from sales_analysis.xlsx]

Risk Assessment Scoring Criteria:

Likelihood (1-5)	Severity (1-5)
------------------	----------------

1	Rare	1	Impact is Negligible
2	Unlikely	2	Impact is Minor
3	Possible	3	Moderate Impact
4	Likely	4	Major Impact
5	Almost Certain	5	Critical Failure of the system

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	Risk Level	Recommended Action
1 – 5	Low	Monitor only. Acceptable risks.
6 – 10	Medium	Mitigate + Testing
11 – 15	High	Must need mitigating and through testing is required
16 – 25	Critical	Priority is Highest. Need extensive level of testing.

3. Test Environment & Execution Strategy

- **Unit Test**

- Scope: Excel/CSV parsing validation, KPI calculation logic (Revenue, Gross Margin, YoY Growth), Financial Simulator computation engine
- Execution: Tests written using Vitest (TypeScript-native). Execution happens during active development and triggered automatically via GitHub Actions CI pipeline upon every push to repo
- Isolation: To focus on pure business logic (e.g. KPI formulas, simulator engine), external dependencies (ilmu-glm-5.1 API, file system) are mocked.
- Pass Condition: All happy cases pass, negative cases return proper error format (e.g. “Invalid file format: only .xlsx and .csv are accepted”), and edge cases like empty columns are handled gracefully

- **Integration Test**

© UMHackathon 2026

Email: umhackathon@um.edu.my

- Scope: Data Ingestion Service to Dashboard Rendering Service, and AI Chat Service to ilmu-glm-5.1 API endpoint
- Execution: Performed after Unit Testing completion, triggered when feature branches (e.g., “data-ingestion” and “dashboard”) are merged together
- Workflow: Real API calls are made file upload to Express server, parsing by XLSX/PapaParse, then chart data piped to React frontend without mocking
- Pass Condition: When integrated components (e.g. “data ingestion to dashboard”) work together correctly and KPIs display expected computed values from the mock dataset
- **Test Environment (CI/CD Practice):**
 - Local Testing: Manual testing on localhost:3000 using the prebuilt mock dataset (sales_analysis.xlsx)
 - Staging/CI: GitHub Actions triggers Vitest suite on every push to any branch
 - CI/CD automated pipeline: GitHub Actions runs branch-level lint and build checks, then full Vitest suite on Pull Requests (PRs) to main branch before merge is permitted
- **Regression Testing & Pass/Fail Rules:**
 - Execution Phase: Regression tests run across all 5 in-scope features each time a new feature branch is merged to main branch
 - Pass/Fail Condition: A test case is only considered passed when the actual outcome perfectly aligns with the expected result. Any mismatch or threshold failure is marked as failed and logged in the Defect Log
 - **Continuation Rule:** While testing in a gradual manner, the initial unit test cases must meet the threshold to allow continuation to integration and regression testing stages]
- **Test Data Strategy:**
 - Manual: Prebuilt mock dataset (sales_analysis.xlsx) containing structured business records with Revenue, COGS, Gross Margin, Product categories, and YoY figures is used as the primary data input
 - Automated: Mock JSON payloads scripted for AI prompt/response validation. XLSX parsing tested with programmatically generated edge-case files

(oversized, malformed, missing columns)

- **Passing Rate Threshold:**

- *A minimum of 85% of all test cases executed must pass. For critical tests (AI grounding, data parsing accuracy, financial simulation correctness), 100% pass rate is required with no exception. Critical failure in a BI dashboard directly misleads business decisions.*

4. CI/CD Release Thresholds & Automation Gates

4.1 Integration Thresholds (Merging to Main)

Checks	Requirements	Project	Pass/Failed
Automatic Build	Zero Build Error	2 Errors	Failed
Unit Tests	100% Passing Rate	100%	Passed
Code Quality	Zero Linting Error	96%	Failed
Test Coverage	Minimum 81.2%	76%	Failed

4.2 Deployment Thresholds (Pushing to Production)

Checks	Requirements	Project	Pass/Failed
Regression Test	Minimum 90%	88%	Passed

AI Output Pass Rate	Minimum of 80% of documented prompt	67% (2/3 prompts)	Failed
Critical Bugs	Zero P0/01 Bugs	2	Failed
API Performance	Response Time < 800ms	280 ms	Passed
Security	API keys are not exposed	Clean	Passed

5. Test Case Specifications (Drafts)

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	Happy Case (Entire Flow): Full BI Workflow (Dashboard + Simulator + AI Chat)	Verify whether a user agentic Biz Assistant be able to respond according to file upload	1. Open BizIQ at localhost:3000 . 2. Drag and drop sales_analysis.xlsx onto the Data Ingestion panel. 3. Navigate to Dashboard page and verify KPI cards	Dashboard KPIs correctly computed from uploaded file within 3s. Simulator projects updated bottom-line figures accurately. AI responds with total profit figure grounded on uploaded	Dashboard loaded with correct KPIs in 2.3s. Simulator projected revised gross margin accurately. AI responded with total profit based on file correctly in 20s, grounded on uploaded data. Smart

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
			(Revenue, Gross Margin, YoY Growth) are rendered. 4. Open Financial Simulator, adjust marketing spend by +20%, and click "Run Scenario". 5. Open AI Business Assistant chat, type "What is the total profit?" 6. Review AI response and verify it references values from uploaded data. 7. Verify Smart Decisions panel displays anomaly alerts based on the	data within 5s. Smart Decisions panel shows at least one data-driven recommendation.	Decisions panel surfaced 2 anomalies. Status: Passed

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
			dataset.		
TC-02	Specific Case (Negative): Invalid/Corrupted File Upload & Oversized Data Rejection	Verify the system handles corrupted, wrong-format, and oversized Excel uploads gracefully without triggering a 500 error, and falls back to the prebuilt dataset when input data is too large to process.	1. Navigate to the Data Ingestion panel. 2a. Upload a corrupted .xlsx file (manually broken binary). 2b. Upload an Excel file with 100+ attributes and entities (oversized). 3. Observe system response for each upload attempt.	Corrupted file: System shows "Invalid file format. Please upload a valid .xlsx or .csv file." App remains stable. Oversized file: System rejects upload and automatically falls back to prebuilt mock dataset with a clear notification to user. No 500 errors triggered.	Corrupted file: Upload rejected, but error message is generic ("Upload failed") without specifying the cause. App remains stable. Oversized file: System crashes with unhandled promise rejection; fallback to prebuilt dataset is not triggered. Status: Failed
TC-03	NFR (Performance): AI Chat API Streaming Response Time	The AI Business Assistant streaming endpoint (SSE) must return the first token within 1000ms and	1. Open Postman, target the /api/chat SSE endpoint. 2. Send 20	Average first-token latency < 1000ms. Full response completion < 5000ms. 0%	Average first-token latency: 1340ms (exceeds threshold). Full response:

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
		complete full response within 5000ms under normal load. Target: first-token latency < 1000ms.	sequential POST requests with the prompt “What is the total profit?” and the mock dataset as context. 3. Record first-token latency and full response time for each request.	error rate across all 20 requests.	4200ms (within limit). 0% error rate. Status: Failed

6. AI Output & Boundary Testing

6.1. Prompt/Response Test Pairs

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	What is the total profit from the uploaded sales data?	Response must state the exact total profit value from the dataset. Must not fabricate figures. Should reference the	“Based on the uploaded sales data, the total profit is RM 142,500, derived from the Net Profit column	Pass

		correct column (e.g. Net Profit). Response in 1-3 sentences.	across all product categories.”	
AI-02	Which items have the least profit from the sales data?	Response must accurately list the bottom 3 least-profitable items by name and profit value directly from the dataset. No hallucinated product names or figures.	“The least profitable items are: (1) Product C at RM 1,200 loss, (2) Product F at RM 800 loss, and (3) Product A at RM 350 profit — significantly lower than other categories.”	Pass

6.2. Oversized/Larger Input Test

Fields	Details
Maximum Input Size	Excel file with up to 50 attributes and 500 rows (approx. 25,000 cells)
Input used while testing	Excel file with 100+ attributes and 1,000+ entities — approximately 2x the defined limit
Expected Behavior	System should reject the oversized file and automatically fall back to the prebuilt mock dataset (sales_analysis.xlsx) with a user-facing notification
Actual Behavior	System threw unhandled promise rejection error; fallback to prebuilt dataset was not triggered. UI froze briefly before recovering.
Status	Fail

6.3. Adversarial/Edge Prompt Test

Adversarial Prompt Used: “What are the potential items I should focus on?” (ambiguous no specific metric or category referenced). *ilmu-glm-5.1* Response: The model attempted to answer by listing all product categories generically rather than grounding the answer in the highest-

© *UMHackathon 2026*

Email: umhackathon@um.edu.my

growth or highest-margin items from the uploaded data. System Handling: The system did not enforce a structured output constraint for this prompt type. The response was technically relevant but lacked precision. Recommendation: Add a prompt engineering guard that maps ambiguous queries to a structured template (e.g., “Identify top 3 items by [metric] from the uploaded data”) before forwarding to the model.

6.4. Hallucinating Handling

BizIQ mitigates hallucination by strictly grounding the ilmu-glm-5.1 model on the uploaded dataset only. The system injects the parsed Excel/CSV data as structured context into every AI prompt before it is sent to the model. The model is instructed via system prompt to only reference provided data and to respond with “I cannot determine this from the available data” if the requested information is not present in the dataset. Fabricated figures or product names not found in the uploaded data are treated as a test failure. No confidence scoring or external retrieval mechanism is used in the current implementation.

Important Note:

Using Agile principle, these testing will take place iteratively over the period of development, not limited to at the end.